

Semester Projekt

Carport

18/12-2025



Links & information

Link til vores Repository på Github.com

<https://github.com/Guacamoleboy/Semesterprojekt>

Link til vores Demo Video

<https://youtube.com>

Link til vores Deployment via Digital Ocean & SSH

<https://fog.guacamoleboy.dk>

Student information

Andreas Rovelt | cph-ar417@stud.ek.dk | Rovelt123

Jonas Meinert Larsen | cph-jl325@stud.ek.dk | Guacamoleboy

Indhold

Indledning	3
Baggrund	3
Forretningsforståelse.....	3
Teknologi	6
Krav.....	6
Domænemodel.....	10
ER-Diagram	12
Navigationsdiagram.....	13
Mockup	15
Arkitektur.....	18
Særlige forhold	19
Udvalgte kodeeksempler	19
Status på implementering.....	23
Kvalitetssikring.....	24
Proces	27
Konklusion.....	29
Bilag.....	30

Indledning

Vi har over de sidste 4 uger arbejdet på en digital løsning til Fog, og deres carport beregninger – både for kunden men også for sælgerens perspektiv. Projektets omfang er at udarbejde en carport beregner i form af hhv. prisestimat, et .svg render visuelt, stykliste og diverse sat via User Stories. Vi har i vores rapport beskrevet vores fremgangsmåde, tankeproces, samarbejde og udarbejdet diverse diagrammer mm. Dette skaber grundlag for vores opgave som helhed.

Baggrund

Baseret på vores møde med kunden – Fog – har vi fundet frem til netop hvad de ønsker, og hvad baggrunden for kontakt er. Fog ønsker en komplet løsning ift. deres online beregninger hvad angår carporte. Det skal digitaliseres og optimeres, så ting som bl.a. stykliste, rendering i .svg format, status mm sker pr automatik, og dermed sparer medarbejderne for tid. Dette resulterer i, at de kan fokusere på det som giver mening for netop Fog – at have god kundekontakt med reelle kunder.

Vi udarbejder en løsning som skaber overblik, fuld transparenthed, og øget frihed for den enkelte medarbejder. Herudover ønskede Martin - fra vores kundebesøg - en form for CMS-system, hvor han og hans medarbejdere kan ændre diverse ting. fremfor at være låste.

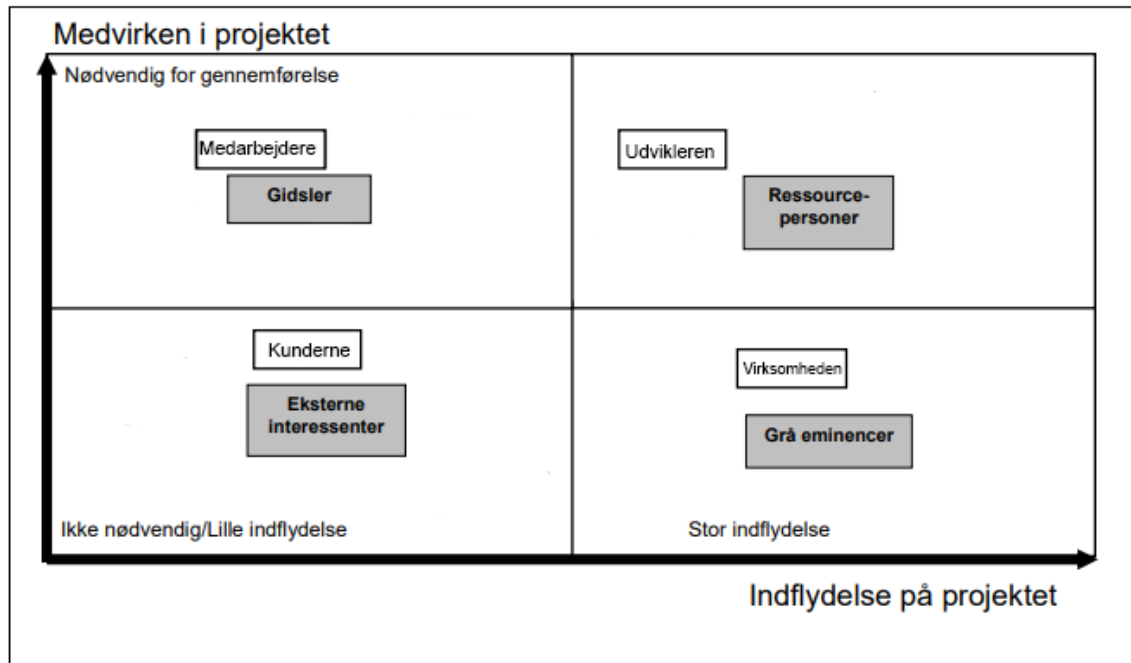
Forretningsforståelse

Ifm. vores projekt har vi kigget og fokuseret på forretningsforståelse som helhed. Dette inkluderer ting som bl.a. interresenanalyse, risikoanalyse og vurderingen heraf, samt gavn af disse ifm. projektet og kundens interesser.

Dette sikrer at vi forstår netop hvad det vil sige at udarbejde et projekt i denne størrelse, og dets omfang.

Interresenanalyse

Vi har udarbejdet en Interresenanalyse som viser et overblik over hvem der har indvirkning på projektet, og kan træffe beslutninger som har betydning.



Figur 1: Analyse af projektets vigtigste interessenter, og deres betydning for projektets fremdrift

Som det fremgår af vores diagram herover, har vi lavet en visualisering af vores interresstantanalyse. Dette sker ved at lave bokse alt efter hvor stor indflydelse de har på projektet. Herunder gennemgår vi dem enkeltvist.

Gidsler:

Medarbejderne har stor rolle ift. gennemførelsen af projektet, da de benytter sig af programmet, og sikrer at det spiller i sidste ende. Det er trods alt dem, som skal bruge programmet dagligt. Derfor sikres det at de er tilfredse, og dermed er de ret nødvendige for projektet og dets fremgang. De har ikke super meget indflydelse, fordi medarbejdere typisk ikke har beslutningskompetence i projektet. Selvom de bliver påvirket af løsningen - og skal bruge den dagligt - er det ofte ledelsen, projektledelsen eller udviklerne, der træffer de centrale beslutninger om krav, prioriteringer og ressourcer. Derfor er deres indflydelse lav, selvom deres medvirken er nødvendig.

Ressourcepersoner:

Udvikleren har høj nødvendighed og stor indflydelse, da programmet bygges af dem. Dette betyder at skulle der ske noget uventet eller problemfyldt med udviklerne eller deres hold, så risikerer projekt sin gang. Derfor er udvikleren en af de vigtigste ift. vores interresstantanalyse.

Eksterne interessenter:

Da vi leverer en digital løsning som laves til kunderne, har de en vis form for nødvendighed, men meget lille indflydelse. Dette sker da kunderne som sådan ikke bestemmer udfaldet af projektet eller dets fremgang. De er dog brugerne, og skal bruge programmet senere. Derfor har de en vis form for indflydelse, da lange load tider og dårlig UI kan resultere i tab af kunder og omsætning. De er også nødvendige, da vi som udviklere ofte kan have svært ved at vurdere features uden reel feedback fra brugeren selv. Derfor er de placeret som de er på vores interresenanalyse. Normalt har man en QA afdeling som tester features og implementeringer inden release, men selve UI brugerfladen vurderes af brugeren, og derfor er de og deres feedback netop vigtig.

Grå eminencer:

Virksomheden har stor indflydelse, da det er deres projekt i sidste ende, og dem som ønsker en løsning og et produkt. De er som sådan ikke nødvendige for projektet og dets afslutning. De er vigtige ift. forventningsafstemning, krav, løsninger og diverse. Derfor er de placeret med stor indflydelse, men lidt nødvendighed i vores interresenanalyse.

Risikoanalyse

Vi har udarbejdet en risikoanalyse over projektet og dets gang. Dette inkluderer ting som sygdom, problemer, hacking og deployment problemer. Vores fulde analyse kan ses herunder, eller i vores mappe "Diagrammer" på vores Github Repository.

Risikoanalyse - Gruppe D				
Risiko	Konsekvens	Sandsynlighed	Forebyggelse	Vurdering
Problemer i udviklergruppen	5	3	Vi sikrer at vi altid har en buffer og laver minimumskravet først	15
Fog lukker ned	5	1	Vi har ingen indflydelse	5
Hacker angreb	5	1	Sikres mod SQL injections mm	5
Sygdom enten i gruppen eller kontaktperson	1	4	Vi sikrer overblik over KanBan	4
Kontaktperson bliver opsagt	1	1	Projektet fortsætter	1
Deployment fejler i flere dage	3	1	Deployment sikres via Digital Ocean	3
Databasen fejler	5	1	Vores database sikres og testes godt af - flere gange.	5

Figur 2: Risikoanalyse over projektet og processen bag

Som det fremgår af vores figur, har vi nogle ret alvorlige konsekvenser på nogen af dem. Vi sikrer os dog mod det, og så er sandsynligheden ret lille for ikke at sige næsten umulig på nogen af dem.

Vores største problem kunne være problemer i udviklergruppen. Dette inkluderer ting som fx uenighed, kommunikationsfejl, folk som dropper ud af studiet mm. Derfor er vurderingen også rød, da den har ret stor indflydelse på projektet og dets gang.

Hvordan har det hjulpet os og vores projekt?

Vi har i vores projekt og gruppe denne gang være ret påvirket af problemer i udviklergruppen. Netop som vi forudså i vores risikoanalyse, som vi lavede inden

projektets start – ugen optil. Vi havde en udvikler som valgte at droppe ud af studiet, hvilket påvirkede dynamikken i gruppen, motivationen og arbejds-mængden. Vi havde dog satset efter minimumskrav til at starte med, og aftalt at vi kunne lave yderligere på senere, så det havde ikke den store indflydelse. Indflydelsen skete mentalt i gruppen, men noget som vi arbejdede med bl.a. med vejleder på i form af Jesper – vores lærer.

Teknologi

Vi har udviklet projektet i Java 17, og benytter os af Maven som build. Herudover har vi benyttet diverse libraries til fx hashing af passwords, http routing osv. Den komplette liste er herunder:

- Javalin 5.6.1
- Jackson Databind 2.15.0
- BCrypt 0.4
- Javalin Rendering 5.5.0
- SLF4J Simple 2.0.7
- Thymeleaf 3.1.3
- PostgreSQL JDBC 42.6.0
- Javax Mail 1.6.2
- JUnit Jupiter 5.8.1
- OpenPDF 1.3.30
- Jackson Datatype JSR310 2.15.2
- XML-APIs 1.4.01
- Batik 1.16 (XML, Transcoder, DOM osv)

Herudover er projektet udviklet i *IntelliJ* samt *Visual Studio Code*, mens der sideløbende er benyttet diverse hjælpeværktøjer uden direkte betydning for projektets funktionalitet. Disse inkluderer fx *Live Server*, *Code With Me*, *Photoshop*, *Figma* m.m., og er derfor ikke medtaget i den tekniske liste.

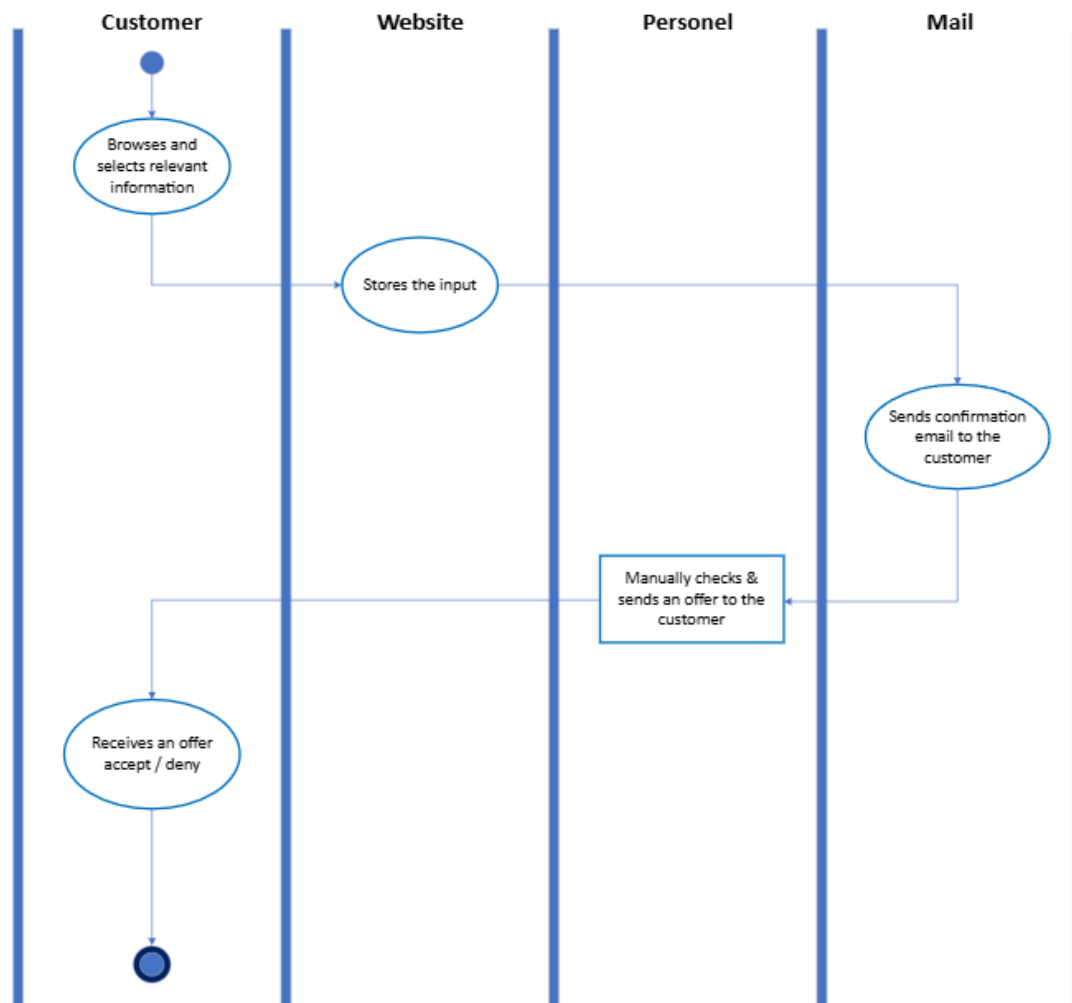
Krav

Ifm. vores projekt har vi ud fra et besøg hos kunden – Martin - fundet frem til nogle krav og ønsker, som implementeringen skal løse. Kunden beder om specifikke forslag, og kommer med problemstillinger som vi finder en løsning på. Vi har her lavet et AS-IS diagram, som viser hvordan virksomheden fungerer på nuværende tidspunkt. Vi har

herudover udarbejdet et TO-BE diagram, som illustrerer hvordan den nye forretningsmodel, og formidling kommer til at være. Alt dette er baseret på User Stories, som er udarbejdet efter et besøg hos kunden, hvor vi finder frem til konkrete løsninger som skal implementeres.

AS-IS

Da vi får kontakt til kunden, prøver vi at finde ud af hvordan den specifikke arbejdsproces er på nuværende tidspunkt. Dette resulterer i et AS-IS diagram, som viser hvordan de arbejder med deres beregner på nuværende tidspunkt.



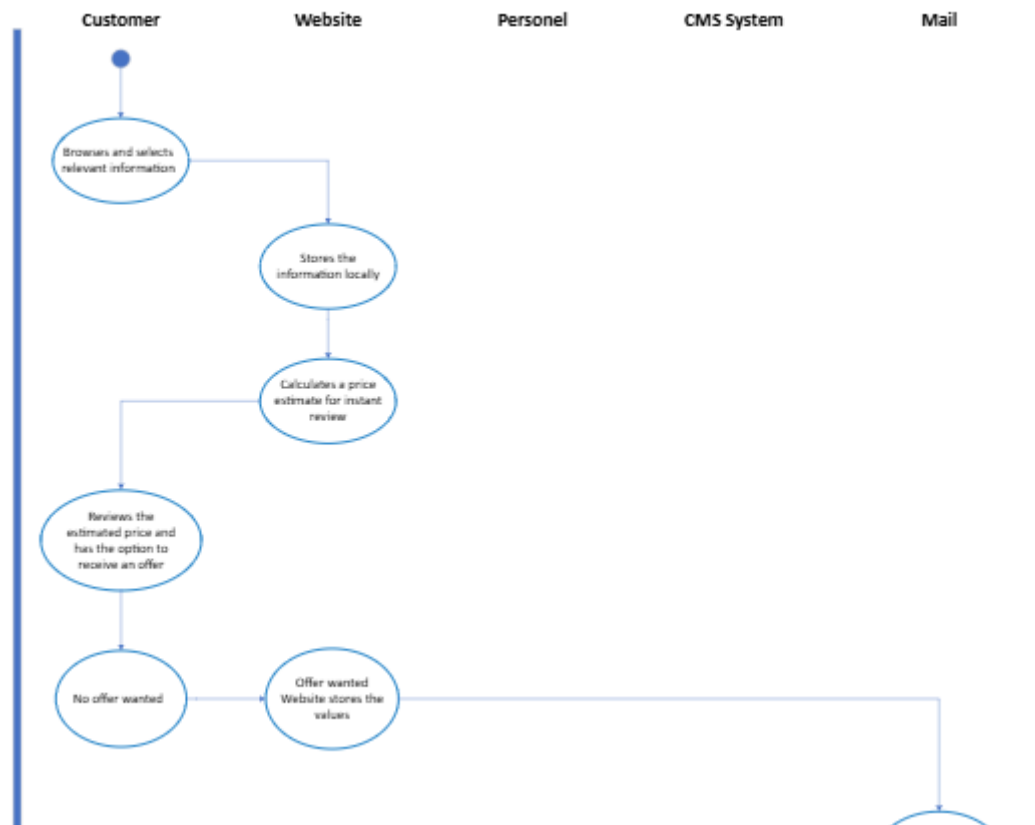
Figur 3: AS-IS diagram over Fogs nuværende løsning

Som det fremgår af illustrationen, skal en sælger ind og godkende og eller justere i hvert tilbud som bliver bedt om. Dette er ikke optimalt, da det må forventes at en stor procentdel af de ønskede tilbud bliver afvist, eller blot er for at få en "2nd opinion" tilbud hurtigt fra anden træhandler. Dette resulterer i unødvendig arbejdstid hos sælgeren for kunder, som med stor sandsynlighed ikke bliver en kunde i sidste ende. Vi

har været inde på Fogs nuværende løsning og teste det af, hvor vi kan konkludere at det netop er sådan det fungerer på nuværende stadie.

TO-BE

Vi har udarbejdet et TO-BE diagram for at illustrere over for kunden hvordan arbejdsprocessen, og formidlingen fra program til kunde bliver efter implementeringen.



Figur 4: TO-BE diagram over Fogs kommende løsning

Vi har i vores TO-BE illustration og håndtering heraf, fokuseret meget på reele kunder, kontra kunder som blot vil have en hurtig pris.

Det er vurderet at sælgerens tid er mere værdifuld hos reele kunder og kundekontakt efter besøget hos Martin hvor han fokuserer meget på, at kundekontakt er hvorfor Fog er Fog.

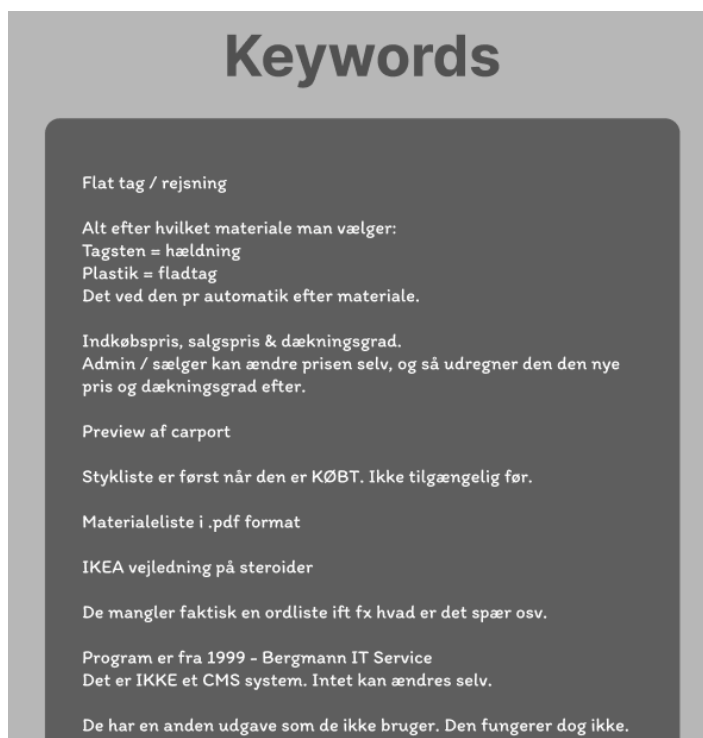
Derfor er beregneren lavet, så sælgeren kun skal fokusere på de kunder som reelt bliver til noget, og de andre kunder får en pris hurtigt som de kan vælge at gå videre med, eller gå til konkurrent. Prisen er vel beregnet og estimeret er præcist. Dette giver kunden en god idé om hvad de kan forvente deres carport skal koste i sidste ende.

Såfremt kunden går videre fra estimatet, vil sælgeren modtage en besked om ønsket tilbud, hvor denne kan oprette et tilbud. Vores styklisteberegner håndterer det meste, men der kan være ting som skal rettes til. Dette kunne være leveringsomkostninger, specialtilbud, justeringer af materiale priser (CMS) og evt. Når tilbuddet er klart modtager kunden en mail, hvor de kan besøge hjemmesiden og se deres endelige tilbud i .pdf format. Dette giver kunden en helt klar idé om hvad deres tilbud indeholder, og priser heraf. Kunden kan vælge at acceptere tilbuddet direkte på siden. Såfremt de gør det, sættes ordren i gang hos Fog.

User Stories

For at kunne komme frem til et TO-BE diagram, er det vigtigt at vi har fastsat nogle User Stories baseret på vores kundemøde. Disse er korte og specifikke krav som skal implementeres for at opfylde kundens ønsker.

Vi har under vores besøg hos kunden skrevet noter og stikord som bruges til at udvikle vores User Stories. Ord som har relevans, og stor betydning for hvad netop denne kunde ønsker for deres løsning.



Figur 5: Keywords ifm opstart af projekt og udarbejdning af User Stories

Vi har ud fra disse stikord dannet os en idé om hvad kundekravet er – specifikt. Dette hjælper både kunden, og os selv med at forstå konkret hvad løsningen skal opfylde.

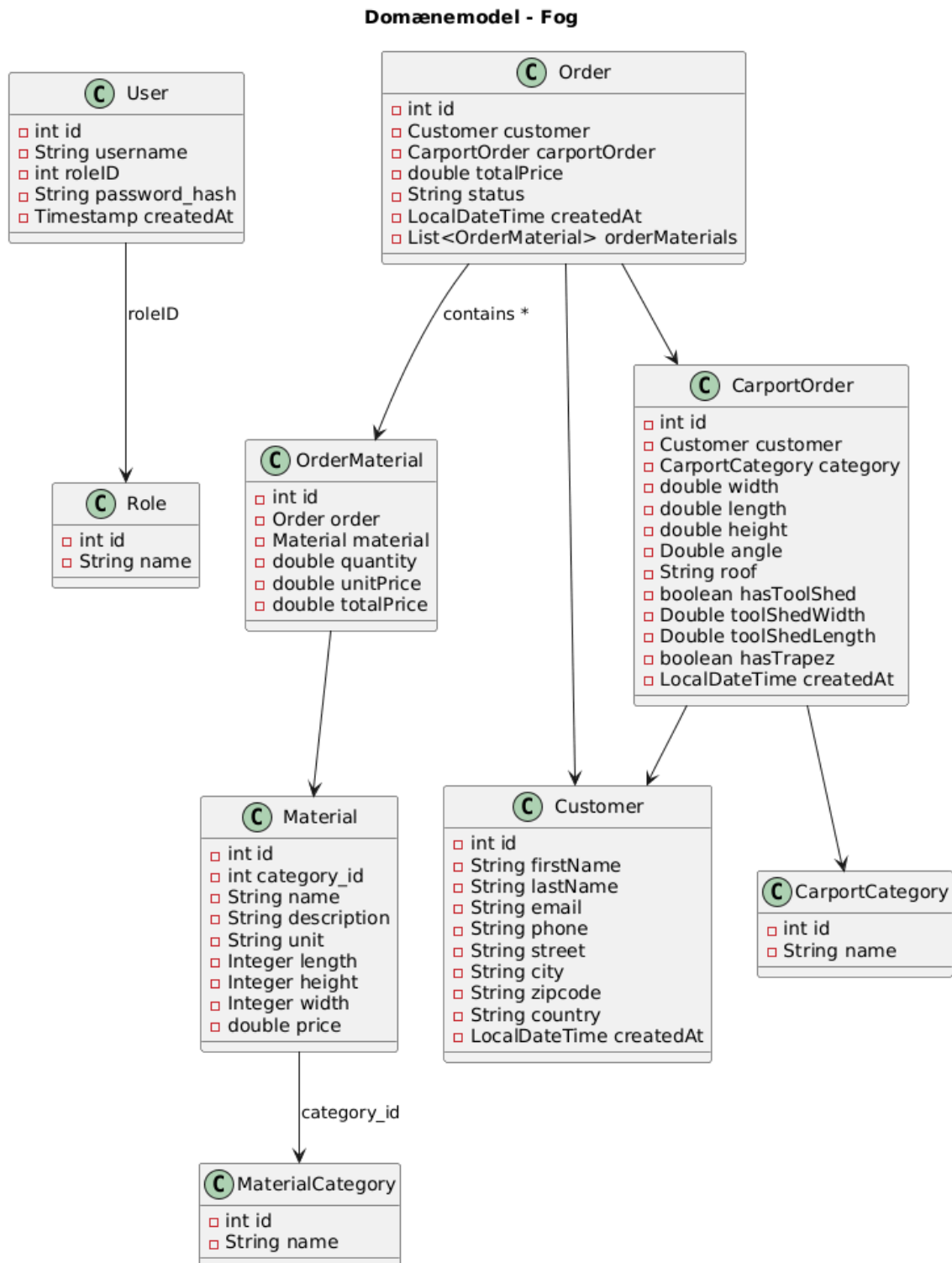


Figur 6: US-1 Overblik – Se bilag 1 for komplet liste

Vi har - baseret på vores keywords - som vi tog mens vi var til kundemøde - udarbejdet nogle User Stories. Disse løser de problemer, og de ønsker som kunden har til projektet. Dette sikrer os at vi opnår netop det kunden ønsker til sidst. Vores User Stories godkendes af kunden i form af User Acceptance Tests, som kan læses mere om længere nede i vores rapport.

Domænemodel

For at etablere en fælles konceptuel forståelse af systemets krav og bevægelser, har vi udarbejdet en domænemodel. Modellen illustrerer de centrale klasser, deres attributter samt de relationer, der binder dem sammen. Dette skaber en klar ramme for design- og udviklingsfasen og sikrer, at alle gruppemedlemmer arbejder ud fra en ensartet systemforståelse.



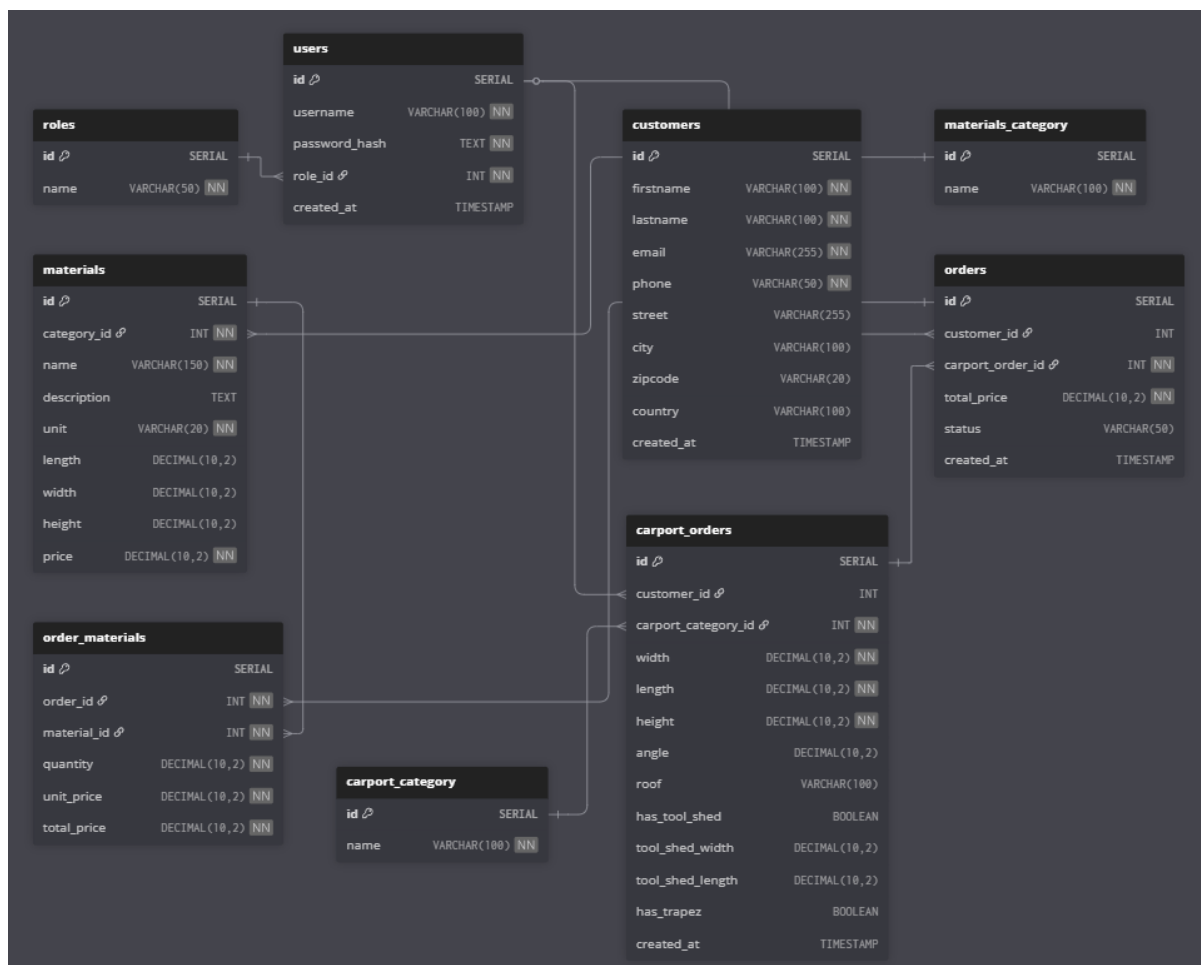
Figur 7: Domænemodel

Modellen fungerer altså som et fælles referencepunkt, der understøtter både kravspecificering, arkitektur og implementering i projektets efterfølgende faser. Modellen viser referencer samt bevægelse i systemet hvilket kan hjælpe os med at forstå – visuelt – hvordan programmet bevæger sig mens vi udvikler på det.

ER-Diagram

For at sikre at alle har samme idé og forståelse for projektet samt databasens indhold, har vi udarbejdet et ER-Diagram. Dette viser relationer og overvejelser ift. projektet, håndtering af data, struktur mm.

Vores database er i 3. normalform.



Figur 8: ER-Diagram

Som det fremgår af vores ER-Diagram herover, har alle vores tables auto-increment på id. Dette sikrer at vi altid stabler data fremfor at overwrite eller potentielt tabe data ved fejl. Når customer oprettes fx, så får customer det nyeste id.

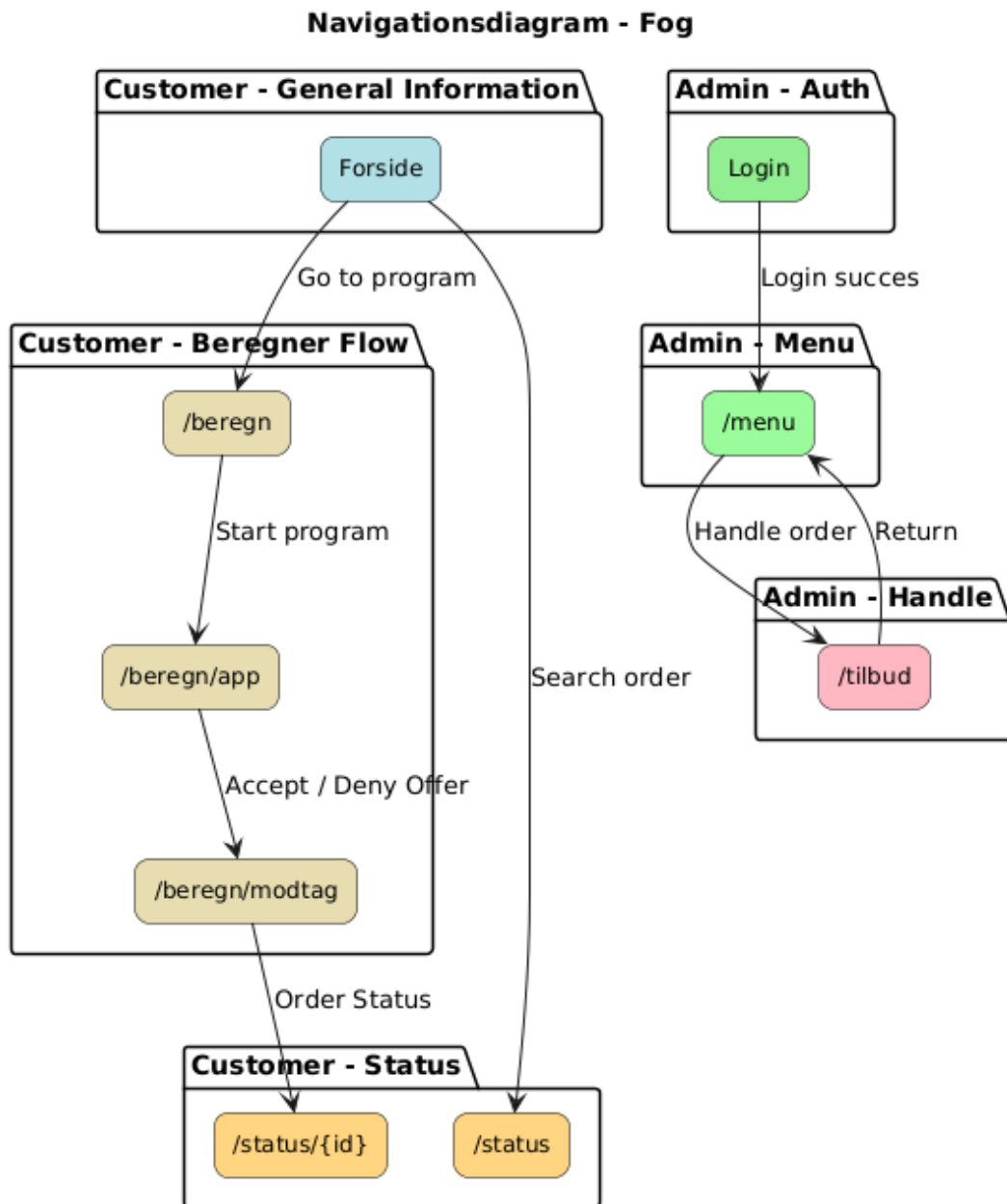
Specifikt for vores ER-Diagram:

- Hver customer kan have flere carport_orders, og hver ordre refererer til en specifik carport_category.
- Orders knytter en kunde til en specifik carport-bestilling, og hver ordre kan indeholde flere order_materials, som hver refererer til en material.
- Materialer er kategoriseret via materials_category, og kategorierne kan indeholde mange materialer.

Ved at kigge på disse og deres relationer har vi mulighed for at sikre at vores udviklere har fælles forståelse, og en idé om hvad der sker i programmet bag facaden. Det kan ofte være forvirrende at danne et overblik ud fra en .sql fil. Derfor er dette diagram essentielt, og en god praktisk løsning at have med i projekterne.

Navigationsdiagram

Et navigationsdiagram kan være effektivt for at vise hvad der sker, og hvor en bruger kan bevæge sig. Vores navigationsdiagram for dette projekt er delt op i Admin og Customer, for at vise, at det er to forskellige løsninger. Admin løsningen er til sælgerne, admin, ledere mm. Disse kan bevæge sig i en vis retning, men har ingen indflydelse på hjemmesiden. Customer derimod kan kun bevæge sig på hjemmesiden, som det fremgår af vores diagram herunder.



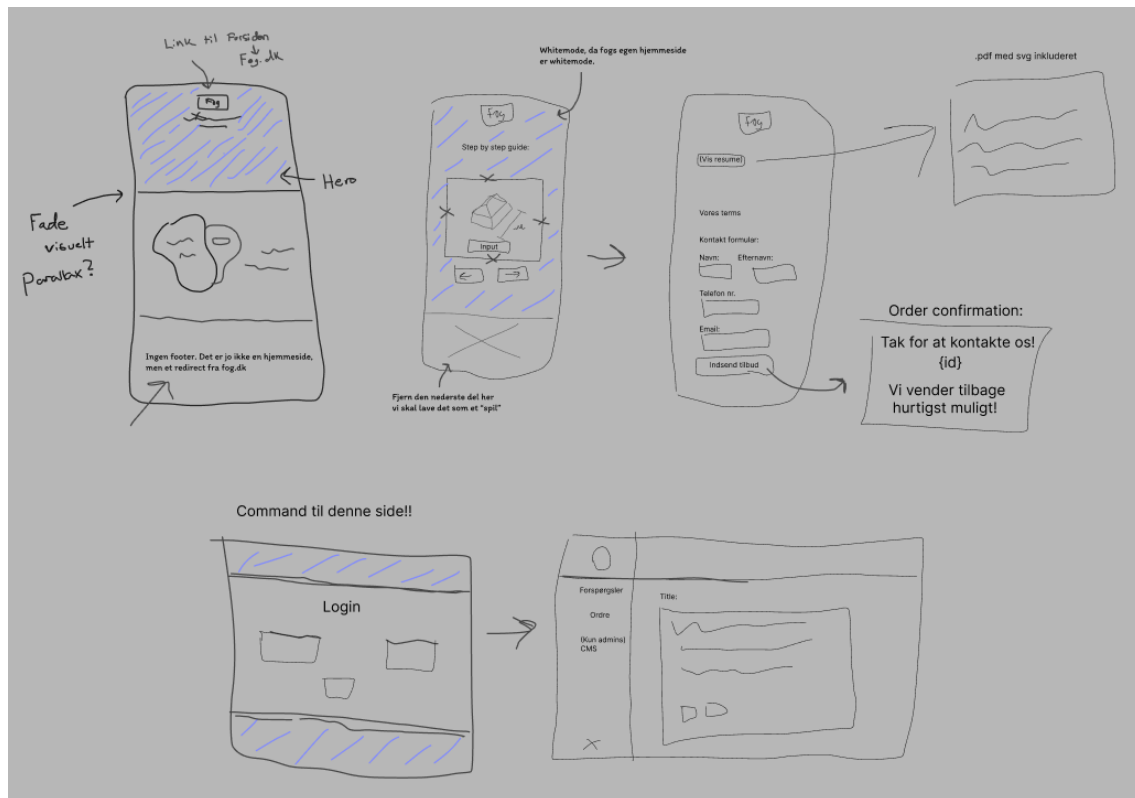
Figur 9: Navigationsdiagram

Programmet er altså delt op for overblikkets skyld, men også for at vise, at det reelt er to forskellige løsninger ift. vores User Stories. Dette er noget der er projekt bestemt, og ikke en generel løsning.

Mockup

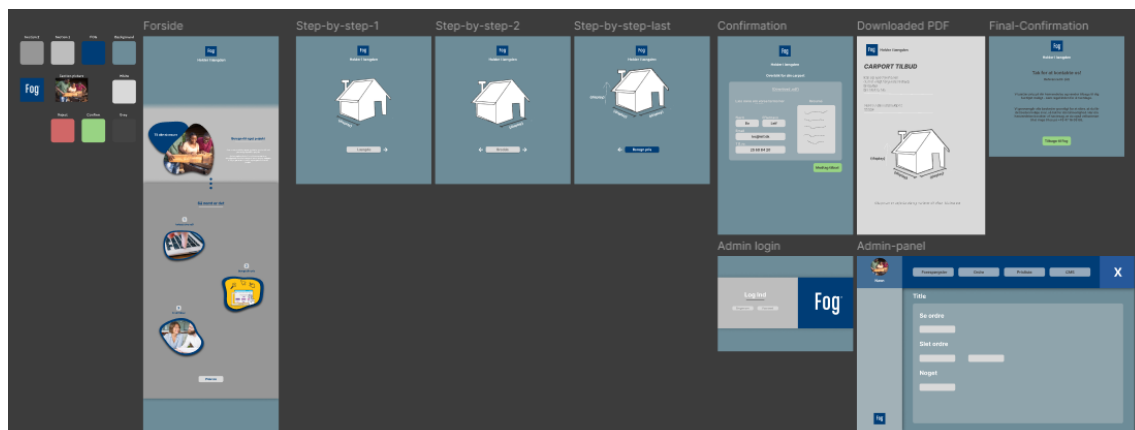
For at skabe en fælles forståelse for projekts visuelle identitet, har vi udarbejdet diverse mockups. Dette sikrer, at vi alle ved hvordan vi vil have at hjemmesiden kommer til at se ud, og spille i sidste ende.

Vi begyndte med vores Paper Mockup som kan ses herunder:



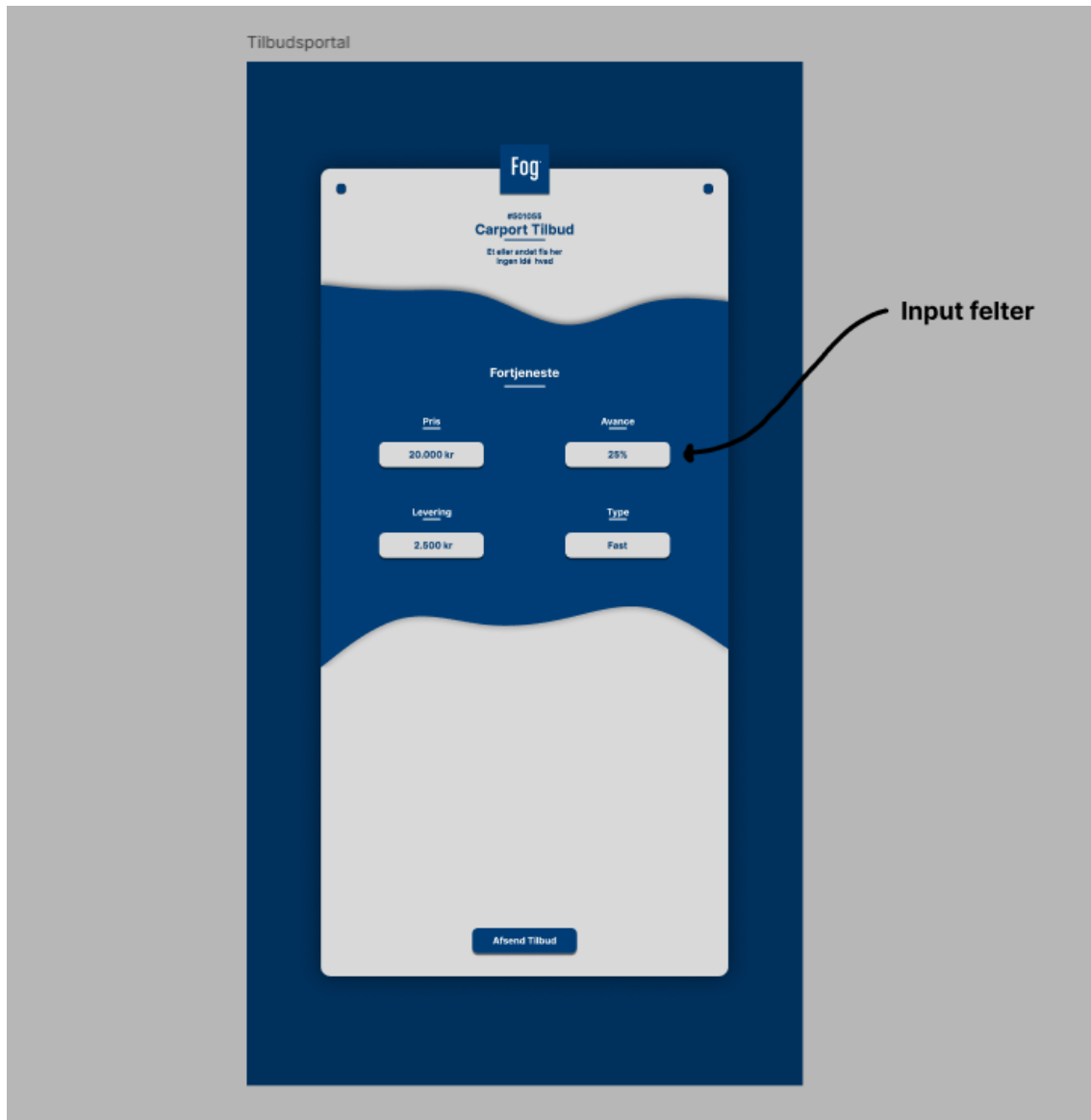
Figur 10: Paper Mockup

Da denne var lavet, gik vi videre til at lave et reelt start mockup, inden vi begyndte at kode. Denne er mere visuel, og giver os en idé om farver og placering af elementer.



Figur 11: Start Mockup

Efter denne gik vi i gang med at kode hvor vi fandt ud af hvad der fungerede, og hvad der ikke fungerede så godt. Dette resulterede i en anden form for mockup – en mere præcis mockup ift. det visuelle.



Figur 12: Mockup afsnit | Tilbud Sektion

Som det fremgår af vores udvikling og forløb, bliver vores mockup mere og mere specifikke og flotte. Dette er baseret på, at vi nu har kodet, og ved hvordan vi vil have at siden kommer til at se ud. Vi kan derfor lave en næsten 1:1 mockup i Figma, inden vi starter på det visuelle med koden.

Herunder resultatet af vores tilbud sektion som hjemmeside element:



Figur 13: Tilbud Sektion

Som vi kan se, er de identiske. Mockups hjælper os med at komme i mål, men også med at designe løsninger på forhånd, sådan at vi bare kan kode det vi ser, uden at skulle tænke for meget i kodesprog.

Det har primært været Jonas som har udviklet frontend delen af hjemmesiden. For ham har det fungeret godt at lave det visuelle først, og herefter udvikle. Det kan godt være vi har en idé i vores hoved, men at ”tegne” det visuelt først, hjælper os med at forstå netop hvad der skal være, og ske på siden. Vi tvinger os altså på en måde til at træffe de beslutninger som danner siden, før vi danner siden. Havde han lavet den uden en visuel approach, så havde der været flere ændringer og tests undervejs, som ikke er nødvendige hvis vi blot laver et hurtigt mockup af siden. For ikke at snakke om øget arbejdstid.

Når det nu kun er én person som laver frontend, så kan man sige – hvorfor så lave en mockup? Er det nødvendigt for projektet? I vores gruppe har vi benytte os af det i stor grad, da Andreas fx primært fokuserer på backend. Han kan kigge på det mockup som er lavet, og så kan han gå i gang uden at skulle afvente frontend alt for meget. Han kender altså knapper, funktioner og det visuelle approach. Dermed står ingen stille, eller afventer nogen eller noget.

Arkitektur

På 2. Semester har vi arbejdet meget med MVC struktureret programmering hvilket er noget vi har taget til os, og benytte os af i høj grad i vores projekt.



Som det fremgår af illustrationen til venstre, har vi opdelt vores program i hhv. controllers, servers, routing og diverse packages.

Dette sikrer totalt overblik over programmet og dets indhold. Herudover sikrer vi, at vi håndterer de forskellige klasser korrekt ift. MVC programmeringsprincipperne.

Vi har controllers i sin egen package, hvor håndtering af *get* og *post* laves og sendes til routing. Disse er opdelt pr side for at skabe overblik over hvilke sider der benytter sig af hvad.

Vores static content loades af vores server configuration via /static. Dette sikrer at alt indhold vi loader på siden, kan hentes med deres respektive packages og navne.

Templates holder vores skabeloner for siderne, som loades med ctx render og Thymeleaf.

Figur 14: MVC Programmering Overblik

Vi har valgt at dele programmet op i mange underfolders og packages, da vi baseret på sidste projekt – cupcake – blev forvirret over hvad der hørte til hvad, trods deres ellers meget brugervenlige navne. Derfor har vi inddelt diverse pr side, så vi ved præcist hvilke controllers der triggers og hvordan.

Det har hjulpet os utrolig meget i dette projekt, og skabt fuldt overblik over opgaven, fremfor at skulle lede efter hvor noget blev kaldt.

Særlige forhold

Som det fremgik af vores ER-Diagram har vi hashing på vores passwords. Dette sikrer, at sikkerheden er i orden, samt at data ikke kan leake uden lige. Vi benytter os af BCrypt som håndterer decrypt ved login, og encrypt når vi opretter en bruger med password.

Vi har laves roles som står for validering af diverse sider bl.a. tilbud og Admin Menu siden. Man kan ikke komme ind på disse uden at være logget ind med en role, som i vores tilfælde er *"sale"* eller *"admin"*. Skulle man forsøge at gå til tilbud uden perms, så bliver man redirected væk.

Eftersom vores Admin menu ikke er tilgængelig for normen, har en registrerings side ingen betydning. Ved aflevering af opgaven sættes nogle admins pre-defined. Disse kan tilføje medarbejdere via vores medarbejderportal. Derfor benyttes kun log ind, og hvorfor SQL injections ikke er det helt store problem. Vi tager dog højde for det.

Udvalgte kodeeksempler

Vi har i dette afsnit valgt at fremhæve kode og teknikker, som vi finder interessante og effektive.

Vi begynder med at fremhæve Jonas' framework – Guacamoleboy Framework. Denne gør – ligesom Bootstrap – at vi har nemt ved at implementere løsninger blot ved at inkludere tags, da det bagvedliggende allerede er lavet. Herudover sikrer det, at vores løsning og responsive design i sidste ende når den skal optimeres til andre skærme, er utrolig nemt at lave.

Vi, samt frameworket arbejder i rem og procent hvilket sikrer, at uanset størrelse på skærmen, har indholdet samme forhold på tværs af skærme.

Hvis vi arbejder i px, så ville det betyde, at 16px fx er 16px uanset størrelse på skærmen. Ved at arbejde i rem og procent, sikrer vi, at vores indhold loader dynamisk baseret på brugeren skærmstørrelse og settings sat i html, body tag.

```
<!-- Status Search Page -->
<section class="status-section guac-d-flex guac-align-center guac-justify-center">
  <div class="guac-container">
    <div class="guac-row guac-justify-center guac-align-center">
      <div class="status-search-box">
        <div class="guac-text-center">
          <h2>Se Status</h2>
          <p>Indtast dit ID herunder for at tjekke status på dit tilbud</p>
          <div class="status-search-wrapper">
            <input type="text" id="status-id" class="status-search-input" placeholder="Indtast ID">
            <button class="status-search-btn" id="status-search-btn">Søg</button>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
<!-- Status Search Page -->
```

Figur 15: Guacamoleboy-framework.css i brug

Som figuren herover viser, benytter vi os af Guacamoleboy-framework via ”guac” tagget. Frameworket består af 1300 linjer CSS-kode med pre-defined settings og diverse components. Hvis vi vil have teksten centreret, kan vi benytte os af ”guac-text-center”, og så slipper vi for at lave text settings på elementet, da frameworket håndterer det bagvedliggende.

Vi har udover frameworket valgt at fremhæve vores måde at loade navbars på. Vi har lavet det som components som loades som scripts. Dette sikrer at vi altid har samme visuals på siderne, og så gør det at vores helikopteroverblik bliver bedre.

Vi skulle løse et problem hvor undersiderne bruger samme navbar, men ikke samme link på ”tilbage” knappen. Det gjorde vi ved at oprette et component i JavaScript, og sætte linket til en variabel, som bliver hentet via id i vores html template.

```

export function loadNavbar() :void {

  const container :HTMLElement = document.getElementById( elementid: "navbar2-component");
  const backLink :string = container.dataset.backlink || "/";
  const navbarHTML :string = `
<!-- Navbar Row -->
<nav class="guac-navbar-overlay navbar-row">
  <!-- Left -->
  <div class="navbar-left">
    <a href="/" class="navbar-logo guac-animate guac-slide-left">
      
    </a>
    <h2 class="navbar-text guac-animate guac-fade-in">Holder i længden</h2>
  </div>

  <!-- Right -->
  <div class="navbar-right">
    <a href="#" class="navbar-right-link guac-animate guac-slide-right">
      <i class="fa fa-phone guac-mr-2"></i> 20 21 44 11
    </a>
    <a href="https://www.johannesfog.dk/om-fog/forretninger" class="navbar-right-link guac-animate">
      <i class="fa fa-map-marker-alt"></i> Find butik
    </a>
    <a href="${backLink}" class="navbar-right-link guac-animate guac-slide-right">
      Tilbage
    </a>
  </div>
</nav>
`;

  container.innerHTML = navbarHTML;

}

// Loads our nav element with the correct link via "data-backlink"
loadNavbar();

```

Figur 16: Dynamisk navbar

Som det fremgår af koden, loader den html kode på siden, men den sætter backLink først som den henter via vores id på siden. Har id intet backlink sat, så er default toggle vores forside.

```

<!-- Navbar -->
<div id="navbar2-component" data-backlink="/beregning"></div>
<!-- Navbar -->

```

Figur 17: Dynamisk navbar i brug via id

Herover ses et eksempel på hvordan vi bruger det hvis vi vil ændre ”tilbage” knappen til noget andet end forsiden. Dette kan vi så gøre forskelligt baseret på hvilken side vi er på. Dermed kan vi lave det hele i ét JavaScript dokument fremfor at oprette samme JavaScript dokument fem gange.

```
public int[] checkWaste(int targetLength, int small, int large) { 3 usages  ⬆ Rovelt123

    int Small = 0;
    int Large = 0;
    int Waste = Integer.MAX_VALUE;

    int maxLarge = (targetLength / large) + 3;
    int maxSmall = (targetLength / small) + 3;

    for (int largeCount = 0; largeCount <= maxLarge; largeCount++) {
        for (int smallCount = 0; smallCount <= maxSmall; smallCount++) {

            int total = largeCount * large + smallCount * small;
            if (total < targetLength) continue;

            int waste = total - targetLength;

            if (waste < Waste) {
                Waste = waste;
                Small = smallCount;
                Large = largeCount;
            }
        }
    }

    return new int[]{Small, Large};
}
```

Figur 18: checkWaste kode eksempel

Algoritmen *checkWaste()* har til formål at finde den bedste kombination af små og store materialer, der tilsammen opfylder en ønsket længde, samtidig med at spild minimeres. Den tager tre parametre, længden på målet samt længden på små og store stykker.

Algoritmen fungerer ved at gennemgå alle realistiske kombinationer af små og store stykker for at finde den løsning, der minimerer spild. For hver kombination beregnes den samlede længde og overskydende spild. Hvis spildet er mindre end det bedste spild, der er fundet tidligere, gemmes denne kombination som den nuværende optimale løsning. Til sidst returneres antallet af små og store stykker, der giver det mindste spild. Algoritmen benytter en "Brute Force" tilgang, hvor alle muligheder tjekkes systematisk, og den bedste løsning løbende opdateres. Dette princip minder lidt om *bubble sort* fra første semester, hvor alle elementer gennem en lykke, sammenlignes og opdateres for at opnå et ønsket resultat, selvom fokus er på optimering af spild - fremfor sortering.

Algoritmen tager ikke højde for pris, selvom det også er en vigtig faktor, da man ofte ønsker en billigere løsning fremfor kun at minimere spild. Dette kan implementeres senere, men i øjeblikket fokuserer vi kun på spild, da indregning af pris også kræver at

tage højde for ekstra skruer og beslag, som eventuelt ville være nødvendige for at samle små materialer til ét stort.

Status på implementering

Vi har leveret en komplet løsning uden fejl eller mangler ift. User Stories samt kundekrav. Denne løsning er klar til brug fra Fogs side af, og håndterer alt lige fra beregning af estimat, til oprettelse af .pdf med .svg render, stykliste og backups af tilbud.

Der er ting som vi klart kunne have lavet bedre, og tænkt over anderledes, men vi føler selv at vi er nået godt i mål med projektet.

Vi er som gruppe ikke så glade for at lave Unit Tests, men vi har over de sidste 2 projekter – denne inkluderet – valgt at prioritere disse fra start af, og tvunget os selv til at lave dem godt og korrekt. Dette lærer os om alvoren, og brugen af Unit Tests. Herudover har Jonas sat Workflow op på Github som kører vores Unit Tests ved Pull Requests, som viser om den kan merges sikkert eller ikke. Dette kan læses mere om senere i rapporten.

Vi har fokuseret meget på UI/UX og principperne heraf. Ting som fx altid at kunne komme tilbage eller væk fra siden. Vi har lavet flot implementering af vores løsninger, samt opdateret den .pdf som bliver genereret via Figma designs lavet af Jonas. Vi er som gruppe splittet mellem om det er nødvendigt at lave det flot eller ikke. Jonas mener det har en stor betydning for projektet og helheden. Det er identiteten, og løsningen skulle gerne afspejle det – mener Jonas. Hvorimod Andreas prioriterer at få logikken i koden til at fungere, før han finpudser det visuelle. Det giver lidt konflikter, og problemer i gruppen internt. Det lykkedes altid, men vi går ikke ind til et projekt med samme idé om et slutprodukt. På baggrund af dette og sidste projekt – har vi valgt, at Andreas sørger for, at vi ikke laver for meget ift. gruppens størrelse, problemer og tidligere erfaring med disse. Det vil sige, at vi lavede vores User Stories, og på baggrund af disse lavede vi Mockups som var ”Minimumskrav”, som vi startede med at arbejde efter.

Jonas har generelt svært ved at begrænse projekternes størrelse, hvilket betyder at vi ender med at lave mere end vi burde. Derfor har vi i dette projekt lavet det anderledes, ved at prioritere det nødvendige først, og herefter kan vi lave videre på andre løsninger og implementeringer.

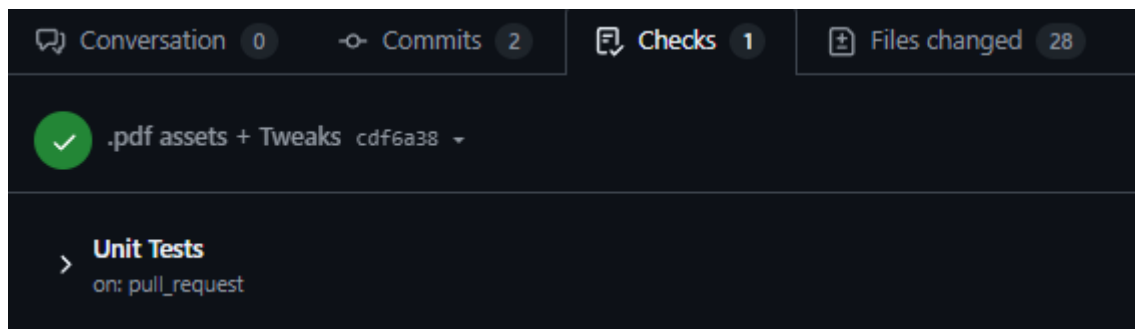
Mht. design på tværs af platform og skærmstørrelser, har vi fokuseret på at kode det til iPhone 12 Pro i Portrait mode for at vise vi forstår hvad vi laver, og hvad man gør for at få styr på design på tværs af skærmstørrelser. Vi har bevidst ikke lavet den til flere størrelser, da vi føler det ikke har relevans, og at vi kunne bruge tiden bedre på andre

ting, features og funktioner. Vi har gjort brug af Jonas' framework – Guacamoleboy Framework – hvilket gør det nemt at justere og redigere i content på tværs af skærmstørrelserne. Det vil sige, at vi med minimal kode kan justere hjemmesiden til at passe til iPhone 12 Pro uden de store problemer.

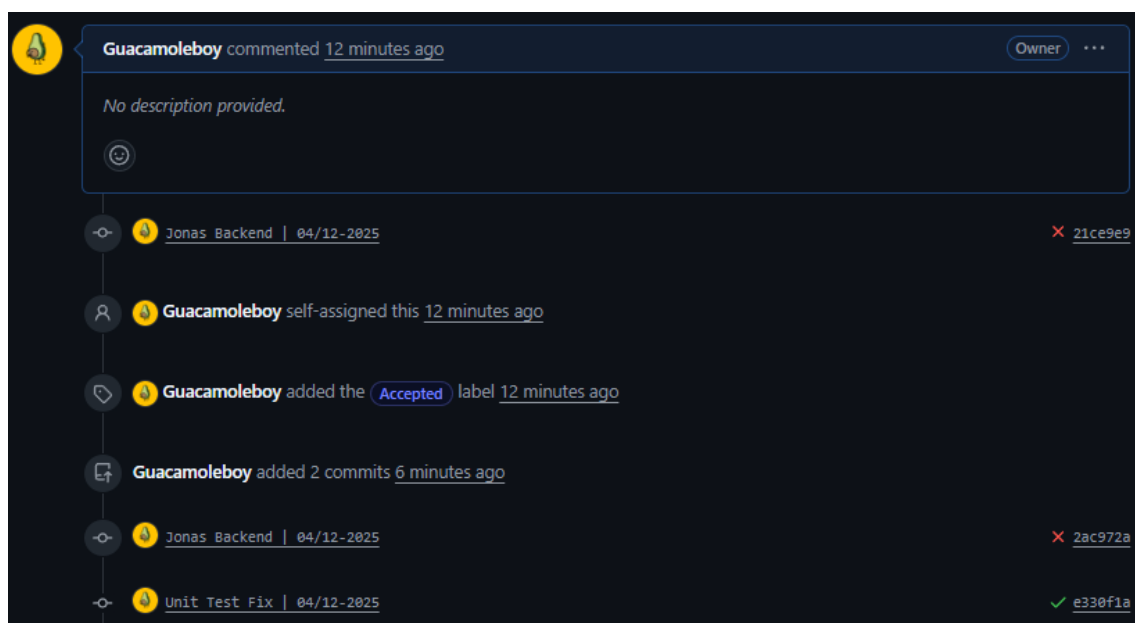
Kvalitetssikring

Vi har i dette projekt prøvet at arbejde med Java CI i form af Github Workflow automatiserede tests ved Pull Requests. Disse sikrer at Unit Tests som minimum kører korrekt. Hvis de ikke gør det, vil programmet fejle, og Pull Requesten bliver afvist baseret på det.

Vores unit tests er vigtige metoder, hvorfor denne ikke må fejle. Herunder en Pull Request som har klaret vores Workflow Tests.



Figur 19: Passed Workflow Test



Figur 20: Failed Workflow Test

På den måde kan man altid ved Pull Request, se om man har lavet en fejl, som skaber problemer for projektet og programmet. Har man det, så må man lave det om, og sikre at Unit Tests fungerer som forventet. Som det kan ses på Figur 19, har Jonas lavet en Pull Request som fejler 2 gange. Dette bliver så rettet til, og til sidst er den så passed, og sikker til at blive merged til vores fælles branch – development.

Vi har til vores Unit Tests lavet vores Database.java klasse om, så vi kan sætte navnet på vores database manuelt i vores tests. Dette gør, at vores Unit Tests kører på sin egen database, og aldrig kommer i konflikt eller til at ændre ved det rigtige data. Er det ikke en Unit Tests, så bruger den bare den normale database.

```
public class Database {  
  
    // Attributes  
    private static String dbName = System.getenv().getOrDefault( key: "DB_NAME",   defaultValue: "fog");  
    private static String host = System.getenv().getOrDefault( key: "DB_HOST",   defaultValue: "localhost");  
    private static String port = System.getenv().getOrDefault( key: "DB_PORT",   defaultValue: "5432");  
    private static String user = System.getenv().getOrDefault( key: "DB_USER",   defaultValue: "postgres");  
    private static String password = System.getenv().getOrDefault( key: "DB_PASSWORD",   defaultValue: "postgres");  
    private static final String URL_TEMPLATE = "jdbc:postgresql://%s:%s/%s";  
  
    // -----  
  
    public static Connection getConnection() throws SQLException {  
        String url = String.format(URL_TEMPLATE, host, port, dbName);  
        return DriverManager.getConnection(url, user, password);  
    }  
  
    // -----  
  
    public static void setDatabaseName(String newDatabaseName) {  
        dbName = newDatabaseName;  
    }  
  
} // Database end
```

Figur 21: Database setup

```

@BeforeAll
static void beforeAll() {
    Database.setDatabaseName("fog_test");
}

// -----

@BeforeEach
void setUp() throws Exception {

    userMapper = new UserMapper();

    try (Connection conn = Database.getConnection();
        Statement stmt = conn.createStatement()) {
        stmt.execute( sql: "TRUNCATE TABLE users RESTART IDENTITY CASCADE");
        stmt.execute( sql: "TRUNCATE TABLE roles RESTART IDENTITY CASCADE");
        stmt.execute( sql: "INSERT INTO roles (name) VALUES ('sale'), ('admin')");
    }
}

```

Figur 22: Unit Test setup

Herover ses hvordan vi benytter os af denne. Udover dette vises det også hvordan vi fx bruger BeforeEach på vores UserMapperTest klasse. Vi sletter data fra users og roles, og så sætte vi roles data efterfølgende. Dette gør, at vores Workflow på Github kan køre tests korrekt, og at vores tests ikke fejler. Satte vi ikke data efterfølgende, så kunne vores UserMapperTests ikke køre, da de benytter sig af roles og dennes relation til hinanden.

Vi har i vores projekt ikke lavet Unit Tests på alle metoder, men kun på metoder som har direkte indflydning på calculations, og hjemmesidens funktioner.

User Acceptance tests

Vi har udarbejdet en User Acceptance Test baseret på vores User Stories. Disse er testede User Stories samt deres status på implementering fra et kundeperspektiv.

User Story	Status
Som bruger kan jeg tilgå siden "Beregn pris" som viser step by step, hvordan jeg skal opmåle min carport, og få en vejledende pris.	Godkendt
Som bruger kan jeg indsætte specifikke anvisninger og materialer, og min tegning samt pris opdateres herefter.	Godkendt
Som sælger skal jeg kunne tilgå en brugers tilbud, og ændre i priser og dækningsgrad for at lave special tilbud til kunder.	Godkendt
Som sælger kan jeg tilgå en menu som kun er for sælgere. Denne indeholder bl.a. tilbud, CMS ændringer af database og anden relevant information.	Godkendt
Som bruger skal jeg kunne se et grafisk preview / visualisering af min carport, så vedkommende får et overblik over det valgte.	Godkendt
Som sælger skal jeg kunne se relevant information som fx dækningsgrad, priser og anden relevant information. Dette opdateres dynamisk ved ændring i tilbud.	Godkendt
Som kunde skal jeg kunne søge efter mit tilbuds ID og se status samt acceptere / ikke acceptere et tilbud i sidste ende som er afsendt af en medarbejder.	Godkendt
Som bruger har jeg først adgang til stykpriser og anden relevant bygge information efter jeg har købt og betalt.	Godkendt

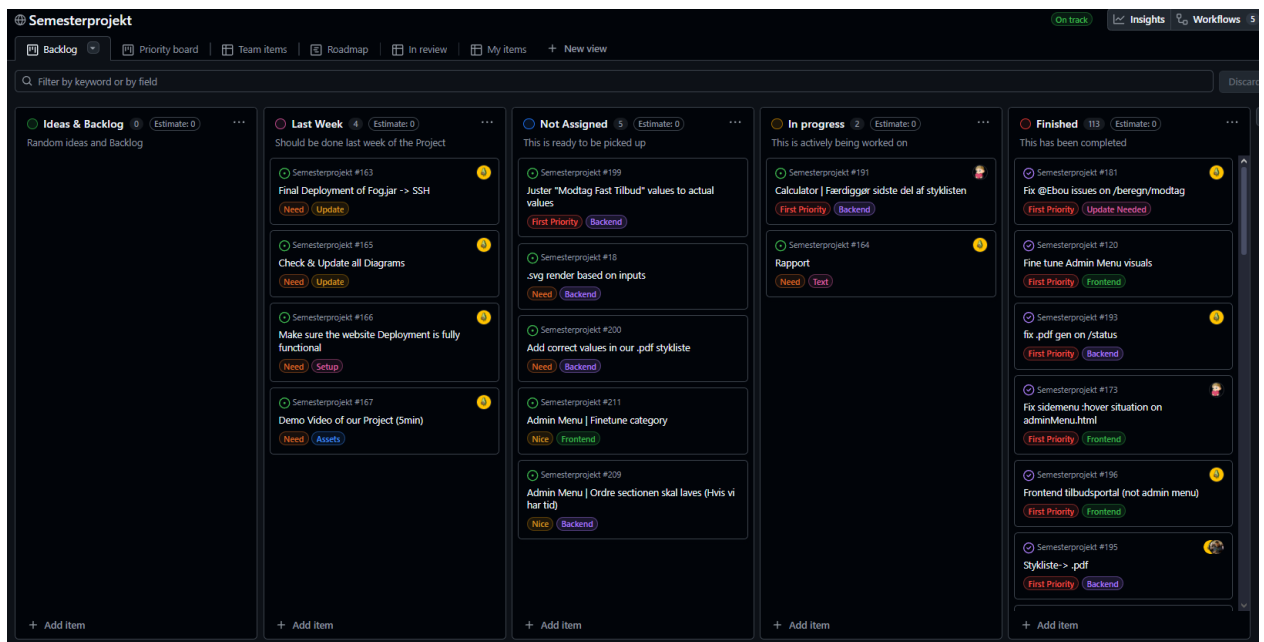
Figur 23: User Acceptance Tests

Som det fremgår af vores figur herover, er vores User Stories implementeret og godkendt fra vores kundes perspektiv – Fog.

Projektet må derfor menes at være nået i mål, og løsning komplet implementeret efter kundens specifikke krav, og ønsker til deres program i sidste ende.

Proces

Vi har i vores projekt arbejdet frit under eget ansvar. Det vil sige, at man laver det man kan, og det man vil. Vi har lavet en KanBan som står for overblikket over de opgaver der skal laves. Dette sikrer at ingen arbejder på det samme på samme tid, samt at de forskellige ting bliver løst. De forskellige issues har fået vigtighed samt category sat på, sig så man ved præcist hvad det specifikke issue omhandler - inden man vælger det. Det skaber overblik, og giver godt indblik i, om det er noget man ønsker at lave på nuværende tidspunkt.

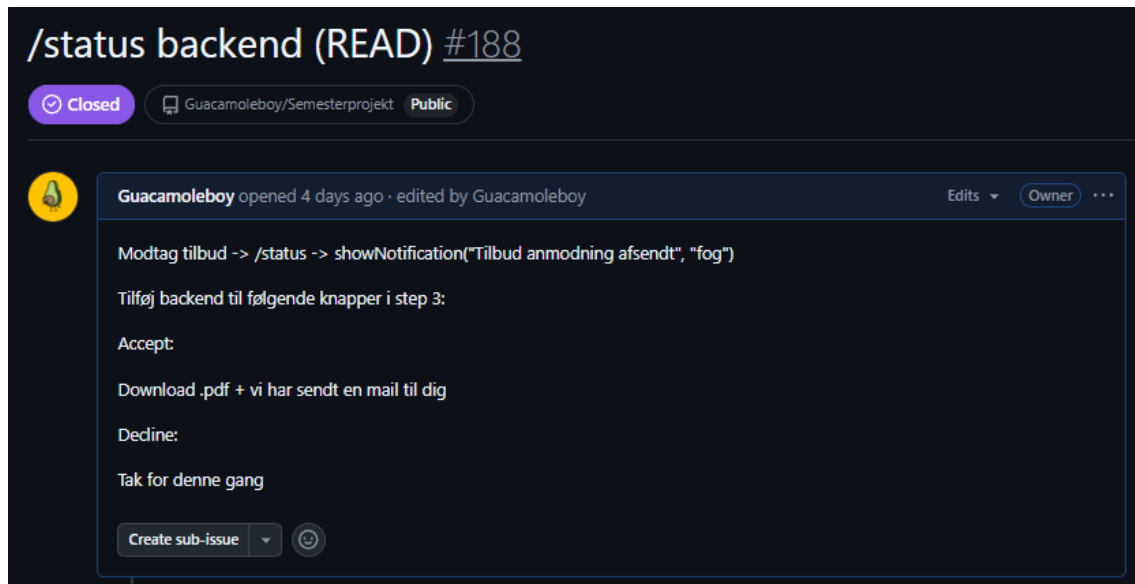


Figur 24: KanBan

Vi har delt det op i forskellige columns, så alle ved hvornår og hvad de laver. Fx har vi "Last Week" som viser hvad vi skal have styr på sidste uge inden aflevering. Dette sikrer at vi faktisk får afleveret alt til tiden. Til sidste projekt glemte vi at sende en Demo Video da det ikke stod på vores KanBan.

Derfor er KanBan et værktøj som vi som gruppe er meget afhængige af, og bruger effektivt og systematiseret fremadrettet.

Mange af vores issues har yderligere information under sig, så vi sikrer at alle forstår netop hvad der sker, og hvad det issue omhandler.



Figur 25: KanBan Issue #188

Skulle en anden end den som har oprettet issueet tage det, så ved personen ned til mindste detalje, hvad der skal ske og implementeres.

KanBan Master (KM) var Jonas i hele projektets periode. Han stod for opdatering af issues fra vores Discord Channel til KanBan overview, samt at skabe overblik over hvad der var løst, og hvad der ikke var løst endnu. Han havde det fulde overblik, hvilket gav frihed til de andre i gruppen, da de ikke havde behov for at have det totale overblik, men blot skulle fokusere på de individuelle issues.

Reflekteret

Vi har som gruppe arbejdet sammen siden studiestart, hvorfor der ikke var de store problemer, med hverken at komme i gang, eller finde en rytme som passede alle.

Vi stødte dog på nogle problemer en uge inde i projektet, hvor Andreas og Jonas blev lidt uvenner, og Ebou droppede ud af uddannelsen. Problemet opstod primært på grund af misforståelser omkring, hvordan vores branches skulle anvendes. Derudover blev noget arbejde slettet af andre medlemmer, fordi det ikke levede op til deres forventninger, hvilket skabte frustration hos dem, der havde udarbejdet koden. Dette skabte splid i en kort periode. Det blev dog løst hurtigt igen blandt Andreas og Jonas, og vi var klar igen til at kode projektet færdigt i god ro og orden.

Vi droppede reviews – hvilket vi startede med at have – ud fra vores død som alle 3 godkendte. Dette fungerede ikke godt for størstedelen af gruppen hvorfor det blev

ændret. Selv da Ebou forlod os fortsatte vi med den opdaterede model, da den nu var implementeret, og fungerede fint i praksis.

Vores estimering af projektet var korrekt – som forventet. Vi har som sagt arbejdet sammen hele studiet, hvorfor vi ved hvordan projekterne løber an efterhånden, og hvor meget vi kan nå at lave. Derfor var det ikke det helt store problem at lave en estimering af projektets størrelse, eller proces heraf.

Vores evalueringsmøder gik primært på den interne arbejdsgang og miljøet i gruppen, fremfor koden eller det at kode. Vi har styr på det med at kode, og fik lavet vores løsning relativt hurtigt. Jesper var sparring og mellemmand, som sikrede at det gik som smurt, og vi undgik flere konflikter end højst nødvendigt.

Deling af koden i Github gik som forventet. Vi har i vores gruppe arbejdet med branches og Github aktivt fra start af studiet hvorfor vi hurtigt har fundet en rytme samt noget som fungerer for os. Personligt bruger jeg – Jonas – Git bash via SSH Key til Github.com. Andreas bruger Github Desktop. Det fungerer helt fint trods vores forskelle, og vi har minimale merge conflicts, som hurtigt kan løses da vi har fælles udgangspunkt i koden. Herudover har Jonas sikret at vi har korrekt .gitignore og diverse checks, for at undgå at unødvendige ting ryger med i Pull Requests, når der bliver brugt Github Desktop.

Konklusion

Som afslutning på projektet må vi konkludere at vores projekt og opgave er løst uden de helt store problemer. Der var lidt med noget risikovurdering som blev til fakta – desværre. Vi fik dog løst opgaven og kom i mål - trods alt - da vi havde forebygget vores problemstilling inden projektet startede, i form af at mindske vores egne krav til opgaven for ikke at lave den for stor skulle der ske uventede problemer internt i gruppen.

Vores løsning og projekt kom i mål, og vi fik implementeret de User Stories og krav som kunden havde. Vi har udarbejdet diagrammer, og kigget på forretningsforståelse – specifikt for Fog, og deres opgave.

Vi har ud fra et kundemøde udarbejdet User Stories selv, og forstår principperne bag samt vigtigheden heraf.

Fremadrettet vil vi fokusere mere på forventningsafstemning i gruppen internt inden projektet, samt undervejs for at forebygge splid. Vi synes dog trods alt at det gik helt fint.

Bilag

Bilag til projektet samt dennes rapport

User Stories

US-1

Som bruger kan jeg tilgå siden “Beregn pris” som viser step by step ^[1]_[SEP] hvordan jeg skal opmåle min carport, og få en vejledende pris.

US-2

Som bruger kan jeg indsætte specifikke anvisninger og materialer, og min tegning samt pris opdateres herefter.

US-3

Som sælger skal jeg kunne tilgå en brugers tilbud, og ændre i priser og dækningsgrad for at lave special tilbud til kunder.

US-4

Som sælger kan jeg tilgå en menu som kun er for sælgere. Denne indeholder bl.a. tilbud, CMS ændringer af database og anden relevant information.

US-5

Som bruger skal jeg kunne se et grafisk preview / visualisering af min carport, så vedkommende får et overblik over det valgte.

US-6

Som sælger skal jeg kunne se relevant information som fx dækningsgrad, priser og anden relevant information. Dette opdateres dynamisk ved ændring i tilbud.

US-7

Som kunde skal jeg kunne søge efter mit tilbuds ID og se status samt acceptere / ikke acceptere et tilbud i sidste ende som er afsendt af en medarbejder.

US-8

Som bruger har jeg først adgang til stykpriser og anden relevant bygge information efter jeg har købt og betalt.